

# Chunked Prefill, Worked Out by Hand

SLICE THE PREFILL, INTERLEAVE WITH DECODE — AND FIND THE SWEET-SPOT CHUNK SIZE

**What this document does.** It takes chunked prefill (DeepSpeed-FastGen, 2024) — the production-ready middle path between cohabitated and disaggregated serving — and derives, from a handful of constants, the chunk size that minimizes decode stall while keeping time-to-first-token (TTFT) inside budget. Instead of processing all 4096 prefill tokens in one forward pass that blocks every concurrent decode for hundreds of milliseconds, we split the prefill into chunks and interleave them with ongoing decode steps. The knob is the chunk size, and picking it is a two-line optimization once you have the formula. Every figure is produced by a reference script (Appendix A, `m04_t04_engine.py`).

**The one idea.** Bigger chunks mean fewer kernel launches and slightly lower TTFT, but each chunk blocks concurrent decode for one *chunk\_time* — the decode stall. The sweet spot is the smallest chunk that stays compute-bound, subject to the TTFT budget. For Nexus that is  $\sim 512$ .

CONCEPTS COVERED, IN ORDER

the cohabitation problem chunking solves; the chunk-count, chunk-time, TTFT, and decode-stall formulas; a full chunk-size sweep; why 512 beats 1024 on the stall SLO; and the arithmetic-intensity floor that rules out going much smaller.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

## Contents

|     |                                                         |   |
|-----|---------------------------------------------------------|---|
| 1   | The problem chunked prefill solves                      | 2 |
| 2   | Step 1 — The four formulas                              | 2 |
| 3   | Step 2 — One chunk size worked end to end ( $c = 512$ ) | 2 |
| 4   | Step 3 — The full sweep                                 | 3 |
| 5   | Step 4 — The floor: why not go smaller than $\sim 512$  | 3 |
| 6   | The argument, at a glance                               | 4 |
| 6.1 | Equation cheat-sheet . . . . .                          | 4 |
|     | Appendix A — Reference implementation                   | 4 |

## 1 The problem chunked prefill solves

Prefill is compute-bound; decode is memory-bound (Module 01). When they share a GPU, they starve each other: a long 6200-token prefill blocks every concurrent decode step for hundreds of milliseconds, so other users' tokens emit in stuttering bursts. Disaggregation (Topic 5) fixes this by using separate hardware pools, but that is only worth it at high volume. Chunked prefill is the middle path: keep one GPU, but slice the prefill so decode can slip in between the slices.

### Intuition

Without chunking, a 4096-token prefill runs as one  $\sim 205$  ms forward pass and *nothing else* on that GPU emits a token for the whole duration. With chunking, the prefill is cut into pieces; after each piece the scheduler runs a decode step for the other in-flight requests, then resumes the next prefill chunk. Decode never waits more than one chunk. The cost is a few extra kernel launches and a slightly higher TTFT for the prefiling request — a trade almost always worth making.

| Quantity                  | Symbol | Value here  |
|---------------------------|--------|-------------|
| Prefill length (p99)      | $P$    | 4096 tokens |
| Prefill throughput (H100) | $R$    | 20000 tok/s |
| Per-chunk launch overhead | $o$    | 0.6 ms      |
| Time per output token     | tpot   | 12 ms       |
| TTFT budget               | —      | 400 ms      |
| Decode-stall SLO          | —      | 40 ms       |

## 2 Step 1 — The four formulas

Everything follows from one set of relations. Let  $c$  be the chunk size in tokens.

### Chunked-prefill TTFT and decode stall

$$\begin{aligned}
 n_{\text{chunks}} &= \lceil P/c \rceil, \\
 \text{chunk\_time} &= \frac{c}{R} \times 1000 + o \quad [\text{ms}], \\
 \text{TTFT} &\approx n_{\text{chunks}} \times \text{chunk\_time} + \text{tpot}, \\
 \text{max\_decode\_stall} &= \text{chunk\_time}.
 \end{aligned}$$

The last line is the crux: a concurrent decode that arrives mid-prefill waits for the current chunk to finish before it runs, so the worst-case stall it sees is exactly one `chunk_time`. The goal is to *minimize decode stall subject to TTFT  $\leq$  budget*.

## 3 Step 2 — One chunk size worked end to end ( $c = 512$ )

**Plug in  $c = 512$** 

$$\begin{aligned}
n_{\text{chunks}} &= \lceil 4096/512 \rceil = 8, \\
\text{chunk\_time} &= \frac{512}{20000} \times 1000 + 0.6 = 25.6 + 0.6 = 26.2 \text{ ms}, \\
\text{TTFT} &= 8 \times 26.2 + 12 = 209.6 + 12 = 221.6 \text{ ms}, \\
\text{max\_decode\_stall} &= 26.2 \text{ ms}.
\end{aligned}$$

Both numbers are comfortably inside budget: TTFT  $221.6 \text{ ms} \leq 400 \text{ ms}$ , and decode stall  $26.2 \text{ ms} \leq 40 \text{ ms}$ . ✓

**4 Step 3 — The full sweep**

Now run every candidate chunk size through the same formulas. Two columns matter: TTFT (must stay under 400) and decode stall (must stay under 40).

| chunk $c$ | $n_{\text{chunks}}$ | chunk_time | TTFT     | verdict                  |
|-----------|---------------------|------------|----------|--------------------------|
| 128       | 32                  | 7.0 ms     | 236.0 ms | ✓ OK (but 32 launches)   |
| 256       | 16                  | 13.4 ms    | 226.4 ms | ✓ OK                     |
| 512       | 8                   | 26.2 ms    | 221.6 ms | ✓ <b>sweet spot</b>      |
| 1024      | 4                   | 51.8 ms    | 219.2 ms | stall > 40 SLO           |
| 2048      | 2                   | 103.0 ms   | 218.0 ms | stall > 40 SLO           |
| 4096      | 1                   | 205.4 ms   | 217.4 ms | stall > 40 (no chunking) |

**Why 512 wins, not 1024**

Notice how *flat* the TTFT column is: from 512 to 1024, TTFT drops only  $221.6 \rightarrow 219.2 \text{ ms}$  — a 2.4ms gain. But the decode stall *doubles*,  $26.2 \rightarrow 51.8 \text{ ms}$ , blowing past the 40ms SLO and reintroducing exactly the TPOT stutter chunking was meant to remove. You would trade 2ms of TTFT for 26ms of stall on every concurrent request. **512 is the sweet spot:** the largest chunk whose stall still clears the SLO.

**Intuition**

TTFT is nearly constant across chunk sizes because the total prefill *work* is fixed at  $P/R = 4096/20000 = 205 \text{ ms}$  regardless of how you slice it; chunking only adds the small  $n_{\text{chunks}} \times o$  launch overhead on top. Decode stall, by contrast, is *linear* in chunk size. So you are not really trading TTFT against stall — TTFT barely moves. You are simply minimizing stall, and the only thing stopping you from going tiny is the next step.

**5 Step 4 — The floor: why not go smaller than  $\sim 512$** 

If smaller chunks mean smaller stall, why not chunk size 64? Because a prefill chunk only earns its keep if it stays *compute-bound*. Arithmetic intensity of a chunk of  $c$  tokens against a  $d \times d$  projection is  $\approx c$  FLOP/byte (Module 01). It must clear the H100 ridge point of  $\sim 295$  FLOP/byte, or the tensor cores sit idle and you have turned prefill back into memory-bound decode.

| chunk $c$ | AI $\approx c$ vs ridge 295 | Regime                               |
|-----------|-----------------------------|--------------------------------------|
| 64        | $\ll$ ridge                 | drifting memory-bound (wasted cores) |
| 256       | $<$ ridge                   | marginal                             |
| 512       | $>$ ridge                   | ✓ compute-bound                      |
| 1024      | $\gg$ ridge                 | compute-bound (but stall too high)   |

**Watch out**

Chunk 128 technically passes both SLOs in the sweep, but with 32 launches and an AI well below the ridge it wastes prefill compute and pays  $32\times$  the launch overhead for no throughput gain. The real constraint is a *band*: stay above the ridge ( $\gtrsim 300$  tokens) for compute efficiency, and below the stall SLO ( $\leq 40$  ms  $\Rightarrow \lesssim 790$  tokens here) for smooth TPOT. 512 is the clean power-of-two that sits squarely in that band, which is exactly why the Nexus v4 config ships `chunked_prefill_size` near 512.

## 6 The argument, at a glance

| # | Step               | Result                                                   |
|---|--------------------|----------------------------------------------------------|
| 1 | Formulas           | $n_{\text{ch}} = \lceil P/c \rceil$ ; stall = chunk_time |
| 2 | Worked ( $c=512$ ) | TTFT 221.6 ms, stall 26.2 ms ✓                           |
| 3 | Sweep              | TTFT flat ( $\pm 3$ ms); stall linear in $c$             |
| 3 | Why not 1024       | saves 2 ms TTFT, costs 26 ms stall                       |
| 4 | Lower floor        | AI must clear ridge 295 $\Rightarrow c \gtrsim 300$      |
| - | Verdict            | $c = 512$ : in the compute / stall-SLO band              |

### 6.1 Equation cheat-sheet

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| Chunk count   | $n_{\text{chunks}} = \lceil P/c \rceil$                             |
| Chunk time    | $\text{chunk\_time} = (c/R) \times 1000 + o$                        |
| TTFT          | $\approx n_{\text{chunks}} \times \text{chunk\_time} + \text{tpot}$ |
| Decode stall  | $\text{max\_decode\_stall} = \text{chunk\_time}$                    |
| Objective     | min stall s.t. TTFT $\leq$ budget                                   |
| Compute floor | $c \gtrsim \text{ridge} (\approx 295 \text{ FLOP/B})$               |

## Appendix A — Reference implementation

Every number above is produced by `m04_t04_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change  $P$ ,  $R$ , the overheads, or the budgets to size the chunk for any workload.

```

1 """
2 Reference implementation for: Module 04 - Topic 4
3 "Chunked Prefill - the TTFT / decode-stall tradeoff, and the sweet-spot chunk size."
4
5 Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6 so the arithmetic in the document is exact and self-consistent.
7
8 Chunked prefill (DeepSpeed-FastGen, 2024) slices a long prefill into chunks and
9 interleaves them with ongoing decode steps. Bigger chunks -> fewer launches, lower
10 TTFT, but each chunk blocks concurrent decode for one chunk_time (decode stall).
```

```

11 We sweep chunk_size to find the value that minimizes stall within a TTFT budget.
12 """
13 import math
14
15 # -----
16 # 0. The Nexus v4 chunked-prefill configuration.
17 # -----
18 prefill_len      = 4096      # p99 prefill length (tokens)
19 prefill_tok_per_s = 20000     # prefill throughput on H100 (tokens/s)
20 launch_overhead_ms = 0.6      # fixed per-chunk kernel-launch overhead (ms)
21 tpot_ms          = 12.0      # time-per-output-token added to TTFT (ms)
22 ttft_budget_ms   = 400.0     # TTFT SLO (ms)
23 tpot_slo_ms      = 40.0      # max tolerable decode stall for concurrent reqs (ms)
24
25 print("== 0. Config ==")
26 print(f"prefill_len={prefill_len} throughput={prefill_tok_per_s} tok/s "
27       f"launch_overhead={launch_overhead_ms} ms tpot={tpot_ms} ms")
28 print(f"budgets: TTFT <= {ttft_budget_ms} ms, decode stall <= {tpot_slo_ms} ms")
29
30 # -----
31 # 1. The chunked-prefill model.
32 #   n_chunks      = ceil(prefill_len / chunk_size)
33 #   chunk_time_ms = (chunk_size / throughput)*1000 + launch_overhead
34 #   TTFT_ms       = n_chunks * chunk_time_ms + tpot_ms
35 #   max_decode_stall_ms = chunk_time_ms (a decode waits one chunk)
36 # -----
37 def model(chunk_size):
38     n_chunks = math.ceil(prefill_len / chunk_size)
39     chunk_time = (chunk_size / prefill_tok_per_s) * 1000 + launch_overhead_ms
40     ttft       = n_chunks * chunk_time + tpot_ms
41     stall      = chunk_time
42     return n_chunks, chunk_time, ttft, stall
43
44 # -----
45 # 2. Sweep chunk_size and check both SLOs.
46 # -----
47 print("\n== 1. Chunk-size sweep ==")
48 print(f"{'chunk':>6} {'n_chunks':>9} {'chunk_ms':>9} {'TTFT_ms':>9} {'stall_ms':>9} verdict")
49 for cs in [128, 256, 512, 1024, 2048, 4096]:
50     nch, ct, ttft, stall = model(cs)
51     ok_ttft = ttft <= ttft_budget_ms
52     ok_stall = stall <= tpot_slo_ms
53     verdict = "OK" if (ok_ttft and ok_stall) else \
54             ("stall>SLO" if not ok_stall else "TTFT>budget")
55     print(f"{'cs':>6} {'nch':>9} {'ct':>9.1f} {'ttft':>9.1f} {'stall':>9.1f} {'verdict}")
56
57 # -----
58 # 3. The sweet spot: smallest stall that still meets TTFT, and meets stall SLO.
59 #   Among SLO-passing sizes, the one with the lowest stall wins (smoothest TPOT).
60 # -----
61 print("\n== 2. Picking the sweet spot ==")
62 candidates = []
63 for cs in [128, 256, 512, 1024, 2048, 4096]:
64     nch, ct, ttft, stall = model(cs)
65     if ttft <= ttft_budget_ms and stall <= tpot_slo_ms:
66         candidates.append((cs, ttft, stall))
67 # Among those that pass, we want low decode stall (smooth TPOT) without paying
68 # much TTFT. 512 is the documented Nexus v4 choice; show the three neighbours.
69 for cs in [256, 512, 1024]:
70     nch, ct, ttft, stall = model(cs)
71     print(f" chunk={cs:4d}: TTFT={ttft:6.1f} ms, decode_stall={stall:5.1f} ms")
72 print(" -> 512 is the sweet spot: TTFT ~222 ms (well inside 400),")
73 print("     stall ~26 ms (inside 40); 1024 trims TTFT by ~3 ms but")

```

```
74 print("    pushes stall to ~52 ms, breaking the concurrent-decode SLO.")
75
76 # -----
77 # 4. Arithmetic-intensity sanity: why ~512 keeps prefill compute-bound.
78 #   A chunk of S tokens at hidden d has AI ~ S (per d x d projection),
79 #   which must exceed the H100 ridge point of ~295 FLOP/byte.
80 # -----
81 print("\n== 3. Why not go smaller than ~512 (compute-bound check) ==")
82 ridge = 295
83 for cs in [64, 256, 512, 1024]:
84     regime = "COMPUTE-bound" if cs > ridge else "drifting MEMORY-bound"
85     print(f"    chunk={cs:4d}: AI ~ {cs} FLOP/B vs ridge {ridge} -> {regime}")
86 print("    Below the ridge the prefill chunk wastes tensor cores; ~512 is the")
87 print("    smallest power-of-two comfortably past the ridge AND under the stall SLO.")
```

— END OF DOCUMENT —